Capítulo 2. Lectura de código

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *El código se lee mucho más a menudo de lo que se escribe.*
>
> —Guido van Rossum, creador de Python

A pesar de la forma en que se enseña la programación , los ingenieros de software dedican mucho más tiempo a leer código que a escribirlo. En la mayoría de los cursos de programación para principiantes, te lanzas inmediatamente a escribir código, centrándote en los conceptos y expresiones idiomáticas básicos del lenguaje, sin tener en cuenta que nunca aprenderías polaco o portugués de una manera similar. Y mientras que la mayoría de los proyectos académicos parten de cero, los desarrolladores en activo casi siempre trabajan dentro de los límites de un código que ha tardado años en llegar a su estado actual, algo que puedes explorar en profundidad en el capítulo 6.

Con la llegada de la programación orientada a agentes o al chat ( ),[1] leer código será aún *más* importante para los ingenieros de software.

Aunque puede que no sea tu primera opción, trabajarás con código que no has escrito tú. No te desanimes, *hay* técnicas que te ayudarán a orientarte cuando te encuentres con código desconocido. En este capítulo veremos por qué leer código puede ser un reto y daremos algunos consejos para simplificar el proceso.

El reto de trabajar con código existente

Independientemente de cómo hayas aprendido a programar, probablemente hayas pasado gran parte de tu tiempo en el maravilloso espacio conocido como *desarrollo greenfield*, donde experimentas el trabajo de empezar desde cero, sin el lastre de trabajos anteriores. Sin embargo, en tu vida profesional, es probable que hayas tenido muy pocas oportunidades de crear una aplicación desde un editor en blanco. Como ingeniero de software en activo, dedicarás gran parte de tu tiempo al*desarrollo de software en entornos ya existentes*, trabajando dentro de los límites de un código base existente y lidiando día tras día con código heredado.

CÓDIGO HEREDADO CON CUALQUIER OTRO NOMBRE...

Leer código antiguo es a menudo una tarea que la mayoría de los desarrolladores prefieren evitar. A menudo utilizamos el término *«código heredado»* para describirlo, y este término rara vez se utiliza como un cumplido. Hay muchas definiciones de código heredado: código que se escribió ayer, código sin una cobertura de pruebas adecuada o con demasiada cobertura, o simplemente código que no escribiste tú.

Sin embargo, no debes menospreciar el éxito de una aplicación existente. Si un producto ha aportado valor empresarial durante años y ha justificado una inversión continua, eso es digno de elogio. Preferimos un enfoque más positivo, como *código heredado* o *código existente*. Para un análisis más profundo del tema, consulta el capítulo 6.

Cuando se te pide que resuelvas un problema relacionado con el código existente, en realidad tienes *cuatro* problemas que resolver.

En primer lugar, debes comprender el problema empresarial que intentas resolver. ¡Y los ámbitos en los que trabajan los desarrolladores son muy exigentes! El software se está apoderando del mundo, lo que significa que los ingenieros de software se enfrentan a retos empresariales cada vez más complejos; gran parte de los frutos más fáciles de alcanzar ya se han recogido.

En segundo lugar, debes ver el problema a través de los ojos del desarrollador que te precedió, y eso suele ser el aspecto más difícil del desarrollo de software. Cada desarrollador resuelve los problemas de

manera diferente; todos tienen sus hábitos y tendencias. Quizás tu predecesor tenía predilección por un estilo de programación más funcional con el que tú no estás tan familiarizado. Quizás utilizó un patrón que es nuevo para ti o aprovechó una biblioteca que no has utilizado antes. Estés de acuerdo o no con el enfoque adoptado, hay muchas formas de resolver un problema determinado.

En tercer lugar, es posible que el código existente no tenga el nivel adecuado de abstracción. Quizás esté modelado de una manera demasiado genérica o no capte los matices adecuados del dominio. Quizás el desarrollador anterior mezcló dos conceptos o impuso un patrón preferido cuando otro habría sido más apropiado. En cualquier caso, es posible que se requiera una refactorización más amplia para que el código sea más comprensible.

En cuarto lugar, al igual que un arqueólogo, a menudo estás desvelando capa tras capa de deuda técnica, tecnologías y enfoques antiguos que ahora pueden considerarse antipatrones. Con el tiempo, los lenguajes y las buenas prácticas evolucionan,[2] y tendrás que ver el código a través del prisma de la época en que fue escrito. ¡Incluso puedes «fechar» el código simplemente fijándote en qué marcos o características del lenguaje se utilizan (o no se utilizan)! Marcos como Spring han evolucionado a lo largo de muchos años, y a menudo admiten múltiples enfoques para un problema determinado. Lo que se considera la forma «correcta» de hacer algo cambia con el tiempo.

Tampoco ayuda el hecho de que casi siempre tengas que lidiar con parches sobre parches. Quizás el último desarrollador no comprendía completamente el problema, o no estaba al tanto de alguna nueva ca-

racterística del lenguaje que podría simplificar enormemente el trabajo en cuestión. Quizás el último desarrollador tenía una licencia premium para un asistente de codificación con IA y generaba 20 000 líneas de código al día. Si a esto le sumamos las típicas exigencias de «arreglarlo rápido», podrías pasar una tarde entera descifrando un solo método.

Trabajar con código existente presenta retos técnicos, por lo que es comprensible que a menudo sea una de las tareas menos favoritas de los desarrolladores. Sin embargo, la aversión va más allá de las cuestiones técnicas e incluye también sesgos cognitivos.

Sesgos cognitivos

Cuando lees código existente, es posible que lo compares desfavorablemente con tu propio trabajo. Por supuesto, *tú* no escribes código malo, ¿verdad? En más de una ocasión, nosotros, tus humildes autores, hemos luchado con algún código, pronunciando variaciones menos educadas de «¿Qué idiota ha escrito esto?», solo para descubrir que lo habíamos escrito nosotros mismos. Y, francamente, si lees código que escribiste hace unos años, *deberías* sentirte un poco decepcionado: es una señal de crecimiento; hoy sabes más que entonces. ¡Eso es algo bueno!

También hay un par de sesgos cognitivos que te perjudican cuando trabajas con código existente. El primero es el efecto IKEA, que se produce cuando le das más valor a las cosas que tú mismo creas. Un estudio descubrió que las personas pagarían un 63 % más por un producto que

hubieran montado ellas mismas con éxito que por un producto idéntico montado por otra persona.[3] Si alguna vez has ido a un huerto de manzanas donde tú mismo puedes recogerlas, a menudo te cobran un suplemento por, bueno, hacer parte del trabajo tú mismo. Sin embargo, lo haces por la experiencia y la oportunidad de seleccionar la mejor fruta directamente de los árboles. En el ámbito del software, los desarrolladores suelen tener opiniones muy firmes sobre la forma «correcta» de hacer las cosas y tienden a preferir su propio código y enfoque.

Además,, existe el efecto de mera exposición: tiendes a preferir las cosas con las que ya estás familiarizado. Esto conduce al dogmatismo típico que muchos desarrolladores tienen en torno a los lenguajes de programación. Los desarrolladores tienden a pensar que el tiempo comenzó con el primer lenguaje que aprendieron. Cuando Java introdujo por primera vez las expresiones Lambda, alguien en una lista de correo específica del lenguaje preguntó por qué Java necesitaba estas «novedosas Lambdas», sin darse cuenta de que las Lambdas no son un concepto nuevo en los lenguajes de programación y que formaban parte del plan original de Java.

Los desarrolladores pueden ser provincianos en cuanto a sus herramientas preferidas, algo que Paul Graham aborda en su ensayo «Beating the Averages» (Superando los promedios). Graham dice que los lenguajes de programación existen en un continuo de potencia, pero que a menudo no se puede reconocer *por qué* un lenguaje es más potente que otro. Para demostrar su punto, introdujo el hipotético lenguaje Blub y un programador Blub productivo. Cuando el programador de Blub mira hacia abajo en la escala de potencia, todo lo que ve

son lenguajes que carecen de las características que utiliza a diario, y no puede entender por qué alguien elegiría una herramienta tan inferior. Cuando mira hacia arriba en la escala de potencia, todo lo que ve son un montón de características extrañas que no tiene en Blub, y no puede imaginar por qué alguien las necesitaría para ser productivo, ya que no están en Blub.

Mientras trabajas con código, así como con tus compañeros desarrolladores, ten en cuenta estos sesgos. Si no estás seguro de por qué un colega se muestra tan inflexible con respecto a una determinada herramienta o enfoque, pregúntale si podría tratarse de un caso del efecto IKEA o de la paradoja Blub. Por supuesto, también debes reflexionar sobre tus propias suposiciones para asegurarte de que tú mismo no estás mostrando ninguna de estas predisposiciones.

Afrontar código desconocido

Por mucho que desees poder dedicar todas tus horas de trabajo a crear código nuevo, a lo largo de tu carrera te encontrarás con bases de código existentes. ¿Cómo puedes ponerte al día en un nuevo proyecto sin perder la cabeza? En primer lugar, empieza por tus compañeros de equipo. Una descripción básica del proyecto debe formar parte de cualquier experiencia de incorporación.

Dedica algo de tiempo a leer la documentación de . Muchos proyectos tienen un archivo README que te ayudará a orientarte, mientras que otros tienen wikis o sitios web diseñados para ofrecerte una visión general concisa.[4] Los proyectos también pueden tener registros de deci-

siones de arquitectura (ADR) .[5] Los ADR proporcionan un contexto muy valioso y el importantísimo «por qué» que a menudo se pierde en la prisa por solucionar el último defecto o interrupción. Puedes aprender más en unos minutos con la documentación que en horas con el depurador. Leer las normas de codificación del proyecto te preparará para los patrones que encontrarás al revisar el código base.

Si la documentación del código que estás leyendo está desactualizada, actualízala a medida que aprendes; si no existe, considera la posibilidad de crear la tuya propia sobre la marcha. Aplica la regla de oro (que se analiza en el capítulo 1). Crear la documentación te ayudará a aprender el proyecto y también será útil para los desarrolladores que vengan después de ti.

Hay muchas cosas que puedes documentar. A medida que leas, escribe tu documentación favoreciendo enfoques ligeros y sencillos que busquen responder a preguntas comunes como estas:

- ¿Qué hace tu servicio?
- ¿Cómo funciona?
- ¿De qué depende?
- ¿Cómo ejecutas la aplicación?

Espera, ya sabemos lo que vas a decir: la documentación puede estar (y a menudo está) desincronizada con el código. Pero créenos, no tiene por qué ser así. La documentación puede evolucionar con el código. La mejor manera de garantizar que así sea es utilizar las pruebas como documentación. Las pruebas escritas con estilos basados en el compor-

tamiento, si se escriben correctamente, pueden producir documentación ejecutable, un tema que se trata con mayor detalle en Capítulo 5.

LAS MÉTRICAS PUEDEN SER ENGAÑOSAS

La cobertura del código (qué parte del código se ejecuta cuando se realizan las pruebas) puede ser una métrica útil en un proyecto. Sin embargo, no hay soluciones milagrosas en el software, y es posible fallar incluso con una cobertura del código del 100 %. Un amigo nuestro se unió a un proyecto que tenía regresiones con cada lanzamiento. Mientras se ponía al día con el código, le preguntó al jefe técnico si había alguna prueba. El jefe técnico respondió con orgullo: «Sí, tenemos una cobertura de código de alrededor del 92 %». Impresionado, nuestro amigo se sorprendió un poco de que tuvieran tantas regresiones, pero continuó con su análisis.

Al examinar el código de prueba, encontró algunos patrones sorprendentes. Al principio, pensó que eran casos aislados, pero finalmente descubrió que eran endémicos en el código base. Volvió al responsable técnico y le dijo: «No he podido evitar fijarme en que tus pruebas no tienen ninguna afirmación». El responsable técnico respondió repitiendo la estadística de cobertura del código.

La lección meta es: desconfía de cualquier métrica, porque pueden ser engañosas. Pero no pierdas de vista el valor y el propósito de una práctica. Si se trata solo de una formalidad, es poco probable que obtengas los beneficios que esperas. Los equipos de proyecto deben desafiarse a sí mismos y a su enfoque con regularidad; no tengas miedo de cambiar de rumbo cuando sea necesario.

Arqueología del software

Una vez que hayas encuestado al equipo y te hayas familiarizado con la documentación existente, es hora de abrir tu editor preferido y practicar un poco de arqueología de software. ¡Arremángate y rebusca en el código base! Parafraseando a Sir Isaac Newton, busca guijarros más lisos y conchas más bonitas. Observa la estructura del código: ¿cómo está organizado? Algunos lenguajes tienen construcciones de primera clase para empaquetar código; otros se basan en convenciones. ¿Cómo encaja el código? ¿Se trata de una arquitectura monolítica o distribuida con docenas o cientos de servicios? ¿Qué conceptos de dominio se expresan en el código? Lee las pruebas: ¿qué te dicen sobre la funcionalidad? Con esa información, ¿entiendes la *intención* de esta clase?

Si la intención no está clara, profundiza más. Los editores modernos pueden hacer que sea muy sencillo ver quién llama a una función determinada, lo que te permite trabajar hacia atrás. Las llamadas deberían ayudarte a determinar qué hace una clase determinada y cómo se utiliza. Tu retroceso puede llevarte hasta un punto final de servicio, como una llamada HTTP, pero al final deberías encontrar la conexión entre una acción determinada del usuario y el código.

Una vez que te hayas orientado, ejecuta la aplicación. ¿Qué hace? Busca un elemento específico, ya sea algo en una interfaz de usuario o un parámetro de una llamada de servicio, y relaciona eso con el código. Mira la lista de problemas; comprueba si centrarte en una sola función o error te permite seguir la ruta de codificación. Busca un punto de referencia; si sabes que una acción determinada da lugar a una actuali-

zación del almacén de datos, búscala en el código. Utiliza tu depurador para recorrer el código: ¿funcionó como esperabas? ¿Terminaste en una ruta de código muy diferente? En última instancia, estás construyendo un modelo mental del código; estás cargando la aplicación en tu cerebro.

¿Cómo se vería esto en la práctica? Usemos la aplicación Spring PetClinic como ejemplo. Incluso si no eres un experto en Java o Spring, navegar por la aplicación debería ser bastante sencillo, además de que cuenta con una excelente documentación. Una vez que hayas clonado y ejecutado la aplicación, verás que tiene la capacidad de buscar propietarios (consulta la Figura 2-1). Si exploras las plantillas de propietarios, verás una con el útil nombre de *findOwners.html*,[6] que hace referencia a un punto final `/owners`. Si buscas `/owners` en el proyecto, obtendrás numerosos resultados, pero un poco de intuición te llevará a la anotación `@GetMapping("/owners")` en el método `processFindForm` del archivo `OwnerController`.

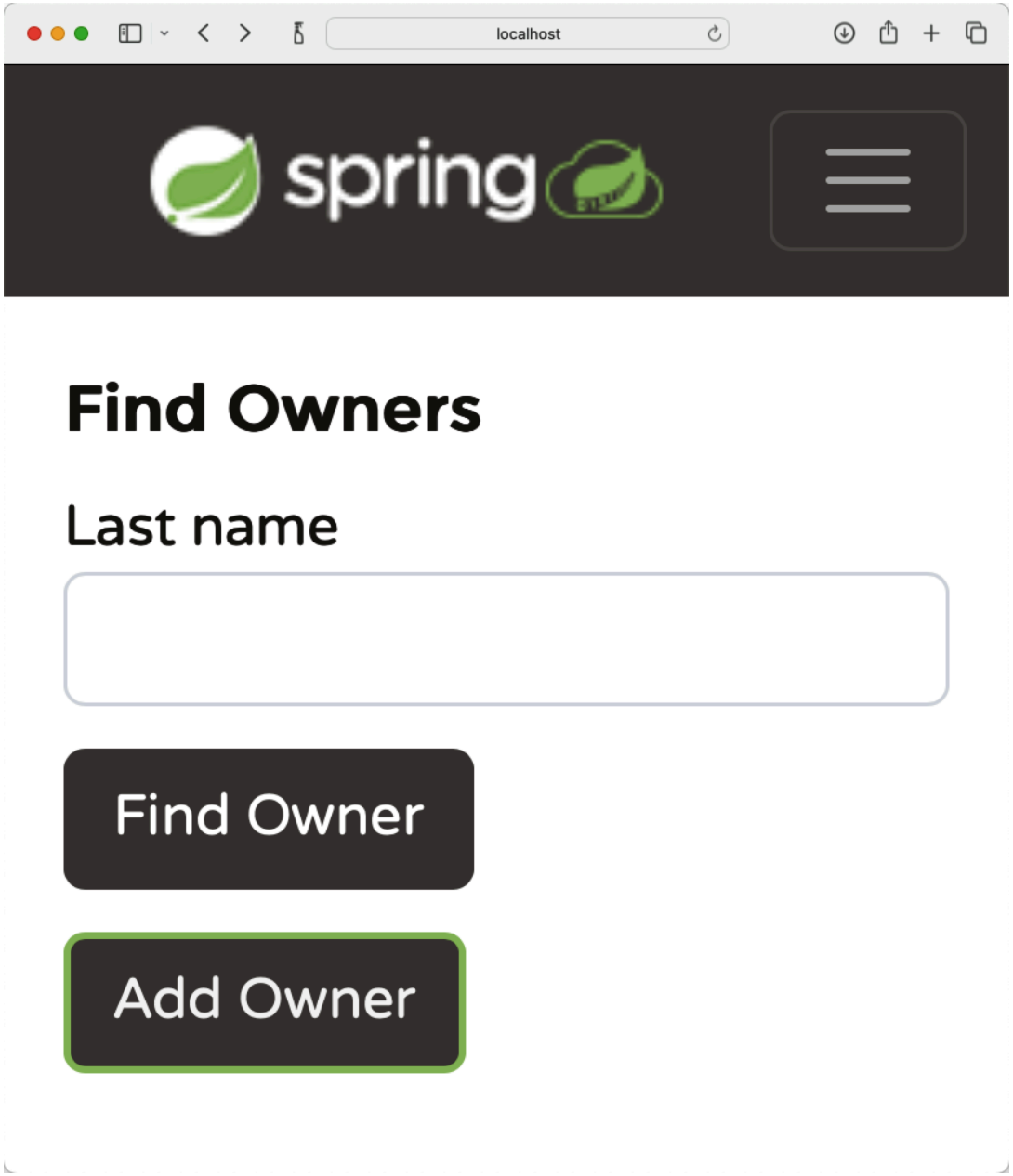


Figura 2-1. Página «Buscar propietarios» deSpring PetClinic

Pon un punto de interrupción, ejecuta la búsqueda desde el navegador y observa qué sucede. Seguramente, tu depurador debería mostrar algo similar a la Figura 2-2. ¿Tu intuición era errónea? Repite los pasos

anteriores. Con el tiempo, establecerás la conexión, lo que te permitirá recorrer el código y desarrollar tu comprensión a medida que avanzas.

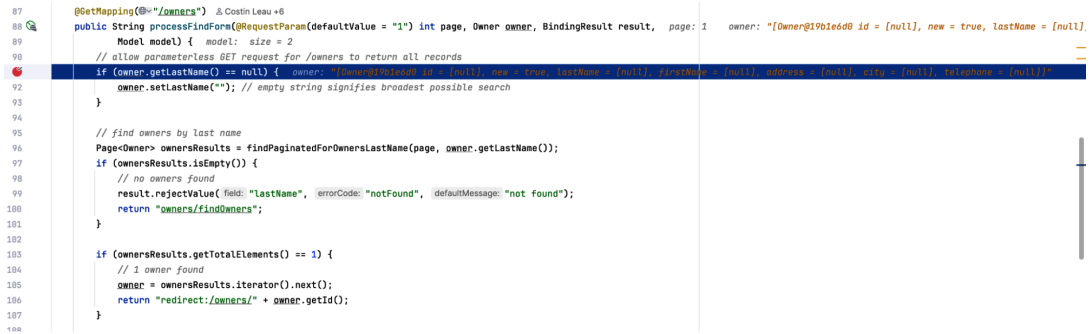


Figura 2-2. Un punto de interrupción en `OwnerController` te permite inspeccionar el estado actual de la aplicación

Utiliza tu entorno de desarrollo integrado (IDE) para navegar por el código. Muchos IDE facilitan el salto a métodos de otras clases (véase la figura 2-3) a medida que avanzas por el código. Considera la posibilidad de contraer todos los cuerpos de los métodos para tener una superficie más pequeña que examinar (véase la figura 2-4). Alternativamente, muchos IDE pueden mostrarte un esquema de una clase determinada, proporcionando una vista de alto nivel del código. Lee los nombres de los métodos. ¿Qué te dicen sobre el propósito del módulo?

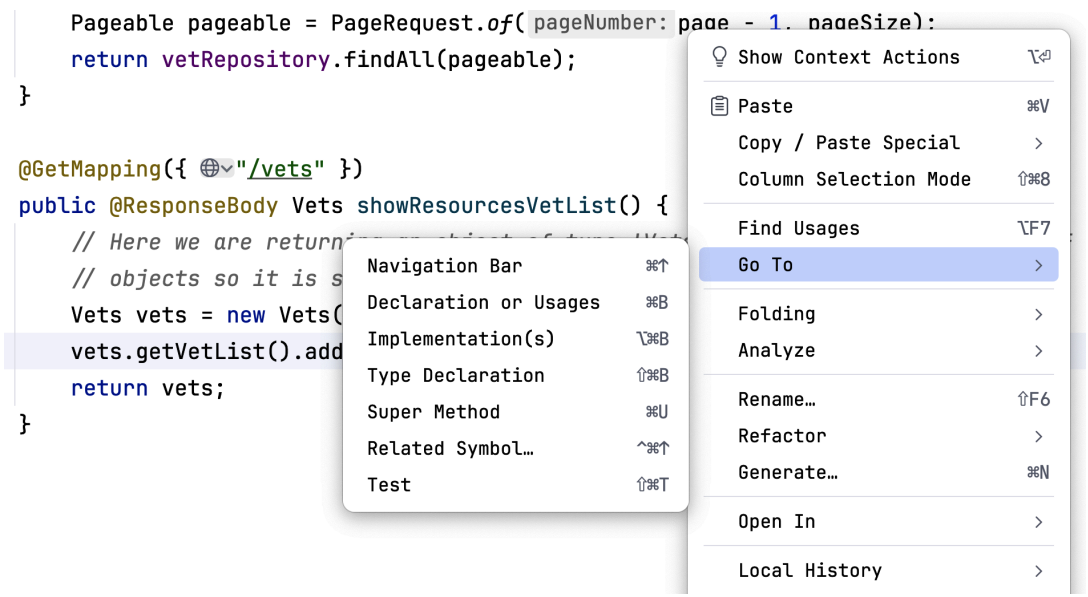


Figura 2-3. Usa tu IDE para ayudarte a navegar rápidamente por el código

```
@GetMapping("/owners/new")  Mic +1
public String initCreationForm(Map<String, Object> model) {...}

@PostMapping("/owners/new")  Mic +4
public String processCreationForm(@Valid Owner owner, BindingResult result) {...}

@GetMapping("/owners/find")  Dave Syer +1
public String initFindForm() { return "owners/findOwners"; }

@GetMapping("/owners")  Costin Leau +6
public String processFindForm(@RequestParam(defaultValue = "1") int page, Owner owner, BindingResult result,
        Model model) {...}

private String addPaginationModel(int page, Model model, Page<Owner> paginated) {...}

private Page<Owner> findPaginatedForOwnersLastName(int page, String lastname) {...}

@GetMapping("/owners/{ownerId}/edit")  Mic +2
public String initUpdateOwnerForm(@PathVariable("ownerId") int ownerId, Model model) {...}

@PostMapping("/owners/{ownerId}/edit")  Dave Syer +5
public String processUpdateOwnerForm(@Valid Owner owner, BindingResult result,
        @PathVariable("ownerId") int ownerId) {...}

/** Custom handler for displaying an owner. ...*/
@GetMapping("/owners/{ownerId}")  Mic +1
public ModelAndView showOwner(@PathVariable("ownerId") int ownerId) {...}
```

Figura 2-4. Contrae los métodos para orientarte en el código base

No des por sentado que el código hace lo que implica el nombre. A medida que el código evoluciona, es posible que los nombres de las variables y los métodos ya no reflejen la realidad. No te precipites; confirma tus corazonadas. Resiste la tentación de tomar atajos y crear potencialmente más trabajo más adelante. En otras palabras, tómate el tiempo que sea necesario, para que te lleve menos tiempo.

Las excepciones pueden llevar a conclusiones erróneas. En más de una ocasión, nos hemos encontrado con excepciones que hacían suposiciones incorrectas sobre posibles condiciones de error. Presta especial atención a las situaciones que detectan excepciones de muy alto nivel; aunque son convenientes para el autor, tienden a ocultar los posibles problemas.

Tu IDE también puede incluir herramientas o complementos que te ayuden a analizar el código. Por ejemplo, IntelliJ IDEA puede mostrarte rápidamente las dependencias, lo que te da una idea de cómo funciona el código en conjunto (véase la figura 2-5). Las herramientas de desarrollo modernas son potentes; deja que te ayuden a comprender el código.

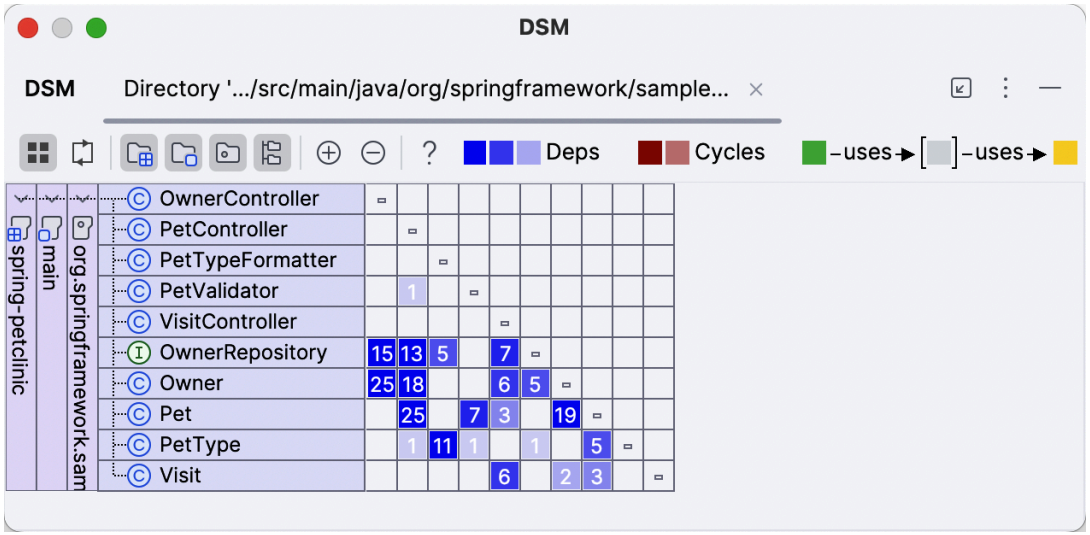


Figura 2-5. Dependencias en el paquete propietario de la aplicación Spring PetClinic

Utiliza también tu herramienta de gestión de código fuente (véase la figura 2-6). Muchas herramientas modernas te permiten moverte rápidamente por tu proyecto. Mira el historial de cambios de los archivos. ¿Qué cambia con frecuencia? ¿Qué te dicen los registros de confirmación sobre las actualizaciones? Empieza por las clases que se modifican con más frecuencia, algo que Git puede mostrarte con un comando como este:

```
git log -pretty=format: --since="1 year ago" --name-only - "*.java"
| sort | uniq -c | sort -rg | head -10
```

También puedes utilizar herramientas como `git blame` para visualizar las modificaciones del código. Si acabas de incorporarte a un nuevo proyecto, ¿quién de tu equipo ha realizado la modificación más reciente o los cambios más frecuentes? Tu IDE también puede mostrarte el historial de cambios si no te apetece utilizar la línea de comandos.

Independientemente de cómo decidas investigar el código, ¡no dudes en preguntar a tus compañeros de equipo si tienes alguna duda!

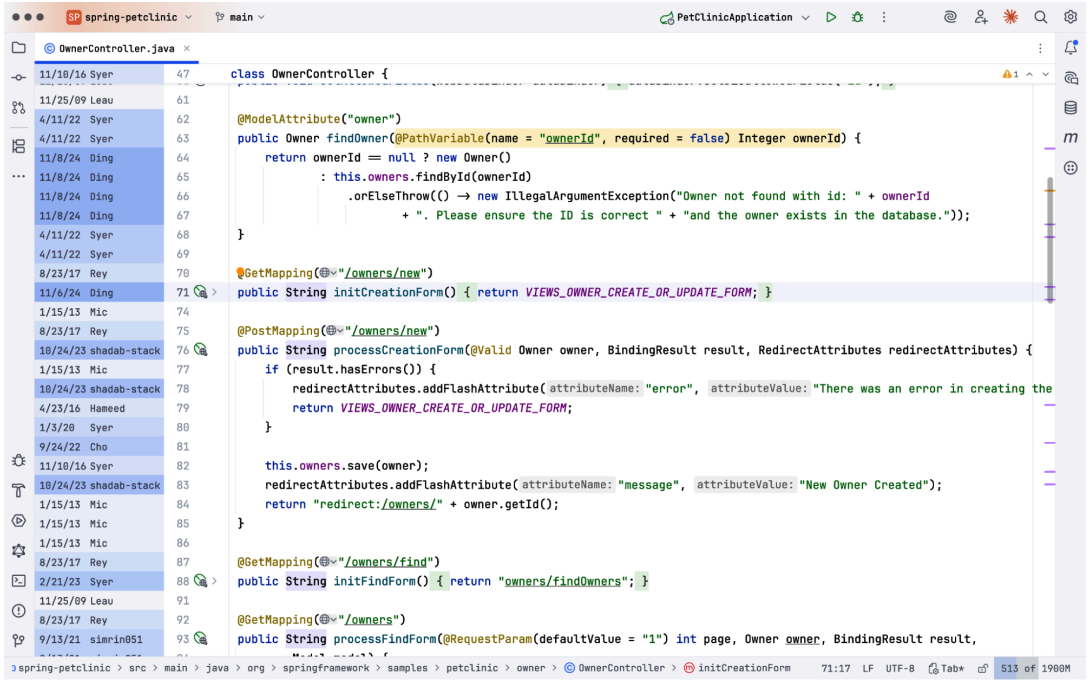


Figura 2-6. Los editoresmodernos pueden mostrar quién editó el código y cuándo lo hizo

Aunque los modelos de e es de proyectos diseñados específicamente suelen desincronizarse con el código a medida que la aplicación evoluciona, siempre puedes *extraer* diagramas del código base. Algunos IDE lo hacen con una simple combinación de teclas (véase la figura 2-7, por ejemplo),[7] pero también puedes utilizar herramientas como Umbrello, Doxygen o Structurizr para crear una representación visual del código. Considera la posibilidad de añadir un paso a tu canalización de compilación que genere automáticamente diagramas nuevos cada vez que se confirme el código en el repositorio.

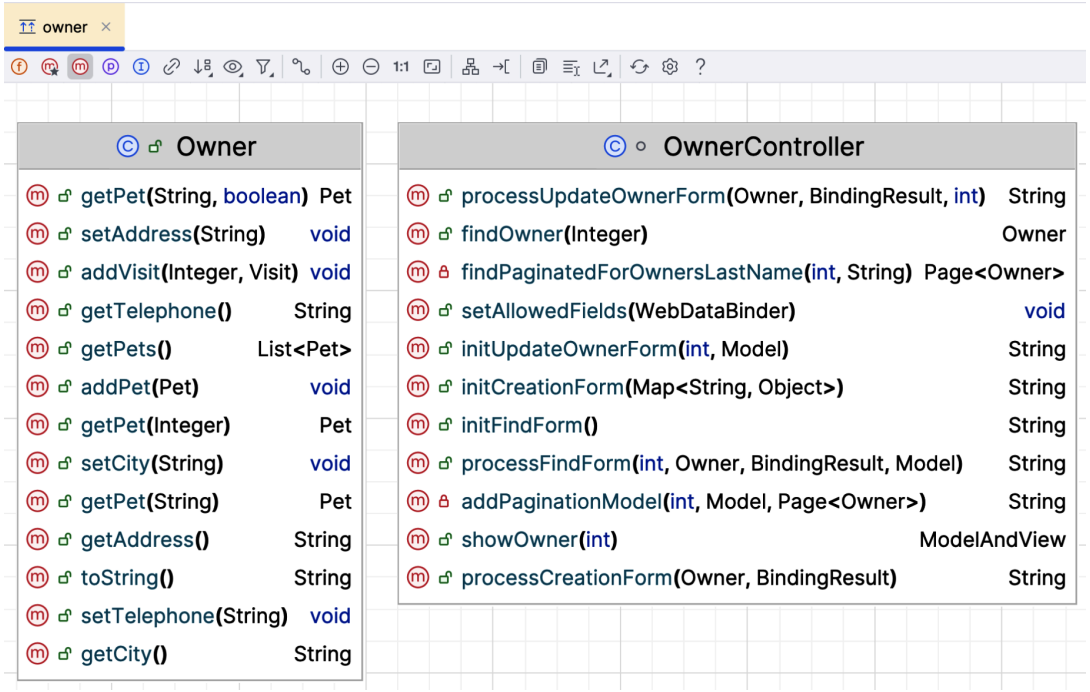


Figura 2-7. Diagramasde clases extraídos del código, que te ofrecen una visión general de alto nivel de la estructura de tus clases

Estrategias eficaces para leer código

Ahora que ya comprendes el propósito del proyecto y tienes una buena visión general, es hora de empezar a entender el código. En esta sección, aprenderás algunas estrategias eficaces para leer código, como aprovechar las funciones del IDE y analizar las pruebas para obtener información.

Aprovechamiento de las funciones del IDE

Tu IDE es mucho más que un editor de texto para escribir código. Los IDE tienen una amplia gama de funciones que pueden ayudarte a escribir, depurar y navegar por bases de código con facilidad. Cuando

trabajas con código existente, tu IDE se vuelve aún más valioso como herramienta de exploración y comprensión.

Enfoque de aprendizaje continuo

Los IDE son tan potentes hoy en día que sería casi imposible descubrir todos los pequeños trucos que tienen. En su lugar, deberías centrarte en aprender algo nuevo cada semana. Por ejemplo, JetBrains IntelliJ IDEA es uno de los favoritos entre los desarrolladores de Java y tiene una función muy interesante llamada «Consejo del día». Cuando abres el IDE, aparece un consejo que te ayuda a familiarizarte con las funciones.

Estos consejos abarcan todo, desde atajos, herramientas, refactorización y depuración hasta complementos. Aprender los atajos de tu IDE te convertirá en un desarrollador mucho más productivo, y este es el primer paso que debes dar. Encontrarás funciones similares en muchos de los IDE que hay actualmente en el mercado, así que consulta la documentación y empieza a aprender.

Si te pierdes el consejo del día en IntelliJ, siempre puedes encontrarlo seleccionando Ayuda → Consejo del día. En el consejo que se muestra en la Figura 2-8, IntelliJ nos enseña una función llamada plantillas en vivo, que se puede utilizar para insertar construcciones de código frecuentes.

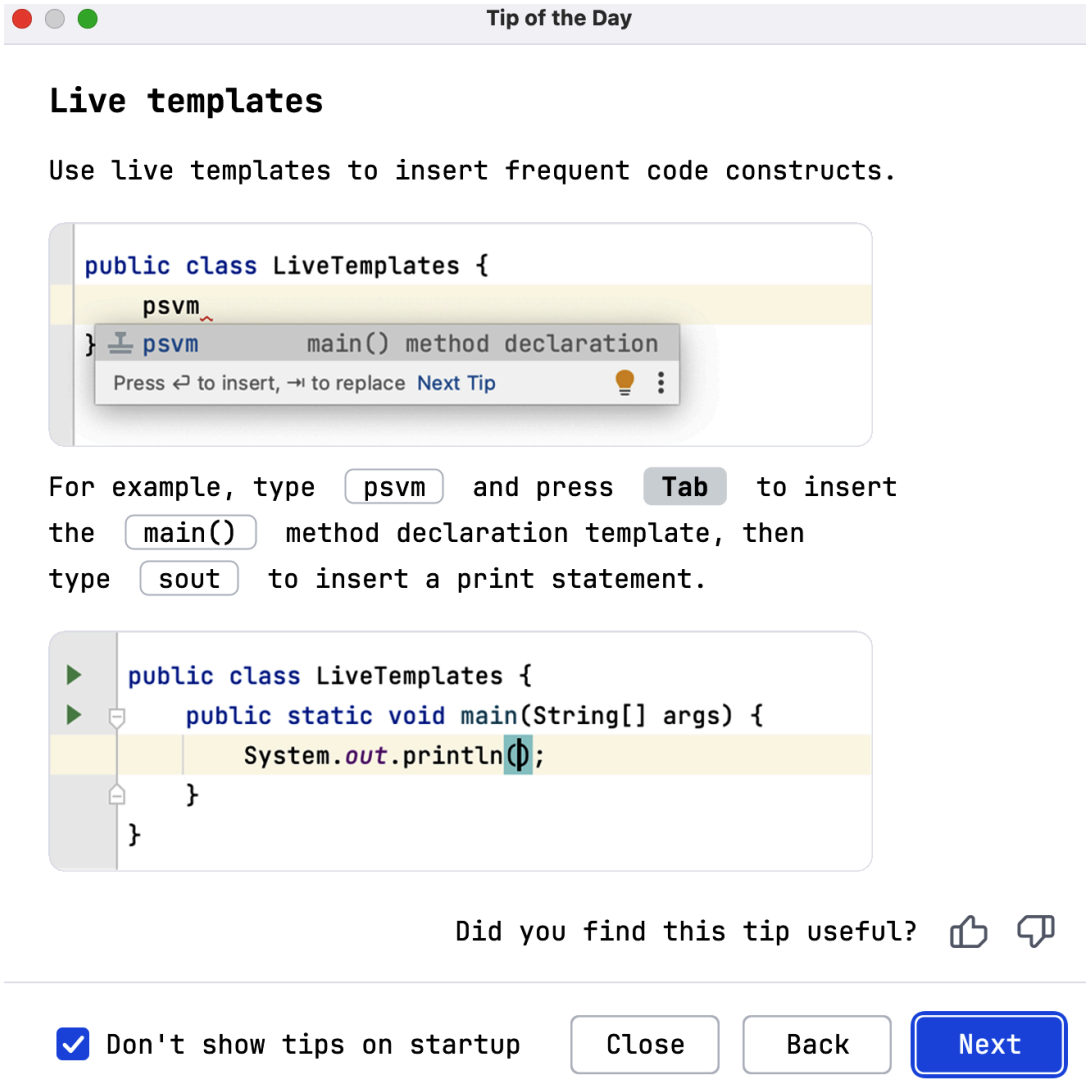


Figura 2-8. Activar el consejo del día en tu editor preferido es una forma sencilla de practicar el aprendizaje continuo.

CONSEJO

Crea un documento personal de trucos de IDE en el que anotes los nuevos atajos o funciones que descubras. Revísalo periódicamente para reforzar lo que has aprendido. Consulta este documento y utiliza los consejos que has anotado para desarrollar esa memoria muscular.

Herramientas de navegación por el código

Una de las habilidades más valiosas a la hora de explorar un código desconocido es la capacidad de navegar por él de forma eficiente. Muchos IDE te permiten saltar e incluso crear código en otros archivos sin tener que abrirlos manualmente a través de la barra de navegación. En las siguientes secciones encontrarás algunas funciones de navegación prácticas que pueden acelerar considerablemente tu exploración del código.

Busca usos y referencias

Cuando intentas comprender una clase o un componente, necesitas ver en qué parte del código se utiliza. Los IDE modernos destacan en esta tarea, y es algo con lo que debes familiarizarte.

Si abrís una interfaz Java en IntelliJ, como la que se muestra en la Figura 2-9, podés hacer clic en el icono del margen para encontrar todas las implementaciones de esa interfaz.[8] Esto resulta muy útil cuando intentas comprender cómo se utiliza una abstracción en la práctica. Del mismo modo, puedes hacer clic con el botón derecho del ratón en el nombre de un método y seleccionar Buscar usos para ver todos los lugares en los que se llama a ese método.

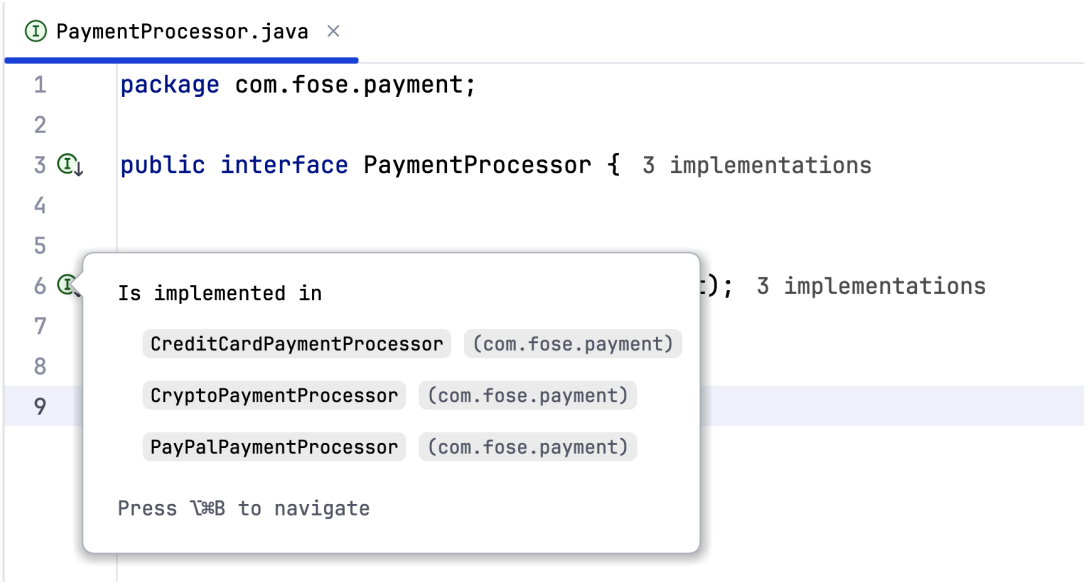


Figura 2-9. Los editoresmodernos pueden mostrar rápidamente dónde se utiliza o se hace referencia a algo y ofrecen la posibilidad de navegar rápidamente hasta ellos

Saltar a la definición

Otra función de navegación útil del IDE es la posibilidad de saltar a la definición de una clase, método o variable. Esto te permite pasar rápidamente del uso a la implementación, siguiendo el flujo lógico del código. Por ejemplo, si estás escribiendo una aplicación Java y utilizas un `ArrayList` y quieres saber cómo funciona, puedes hacer clic con la tecla Comando (o Ctrl en Windows/Linux) en el tipo `ArrayList` para ir al código fuente, donde podrás ver el código y la documentación de ese tipo (véase la figura 2-10).

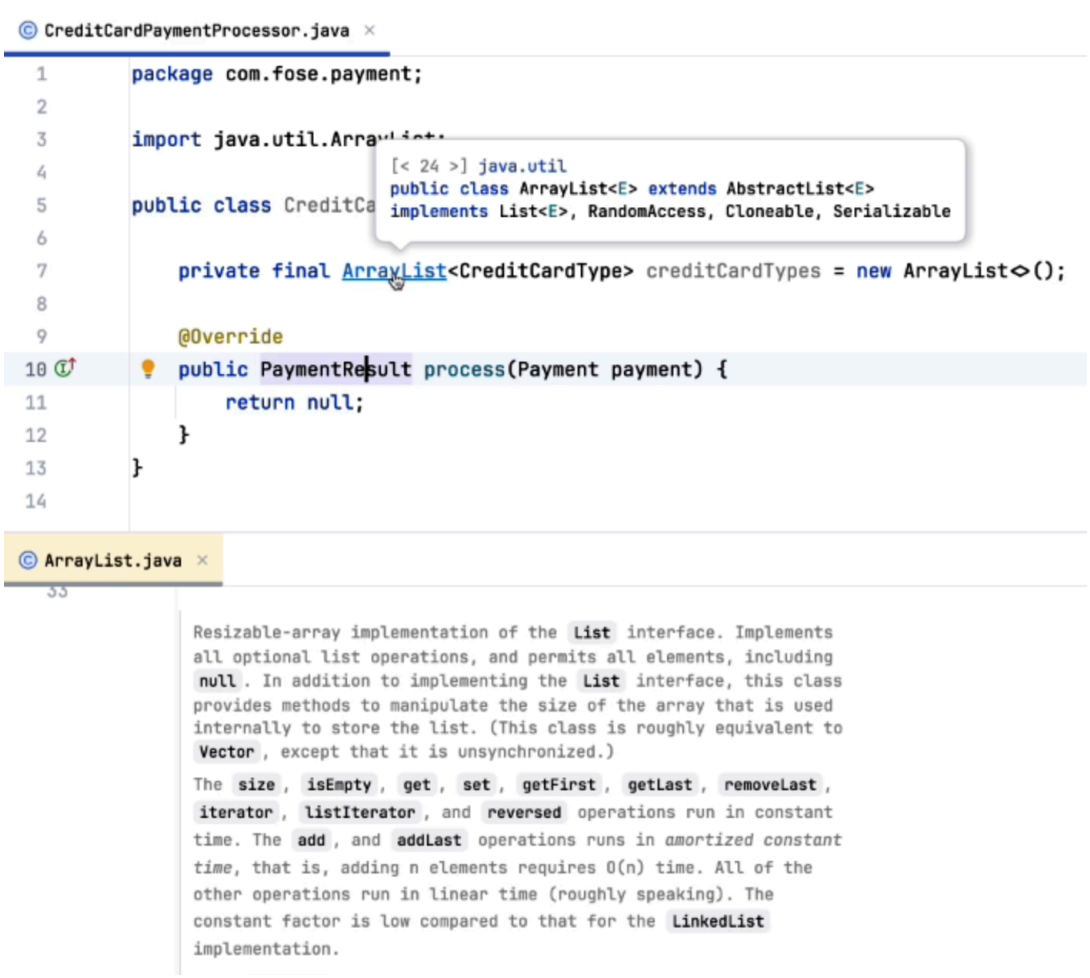


Figura 2-10. Saltar a una definición puede ayudarte a navegar rápidamente por el código

La capacidad de saltar a una definición es muy útil cuando se trabaja con lo siguiente:

- Bibliotecas de terceros
- Código del marco
- Clases base e interfaces
- Métodos de utilidad utilizados en todo el código base.

Jerarquías de llamadas y vistas de dependencias

Comprender cómo interactúan los componentes es fundamental para entender un código base desconocido. Los IDE ofrecen vistas especializadas para visualizar estas relaciones:

Jerarquía de llamadas

: Muestra qué llama a un método y a qué métodos llama.

Jerarquía de tipos

: Muestra las relaciones de herencia

Diagrama de dependencias

: Visualiza cómo los módulos o paquetes dependen unos de otros

Por ejemplo, en IntelliJ, puedes hacer clic con el botón derecho del ratón en un método y seleccionar Mostrar jerarquía de llamadas para ver tanto las llamadas entrantes (desde donde se llama a este método) como las salientes (métodos a los que llama este método). Esto te permite ver rápidamente la función de un método en el sistema general.

Funciones de análisis de código

Los IDE modernos cuentan con potentes herramientas que te ayudan a navegar por el código, pero pueden hacer mucho más. Pueden analizar activamente el código que estás escribiendo y ofrecerte información que puede mejorar tu código y ayudarte a aprender nuevas funciones que quizá no conocías.

Inspección y sugerencias automatizadas

Cuando escribes o modificas código, los IDE pueden ofrecerte sugerencias sobre cómo refactorizarlo. Es importante recordar que se trata simplemente de sugerencias: no siempre tienes que aceptarlas, pero es útil revisarlas. Estas sugerencias suelen revelar patrones comunes y buenas prácticas que no solo pueden mejorar la calidad del código, sino que también te enseñan algo nuevo en el proceso.

Aquí tienes un ejemplo en Java en el que el código itera sobre una colección de `Books`. Para cada `Book` de la colección, el código simplemente imprime el nombre del libro:

```
List<Book> books = library.getBooks();
for (Book book : books) {
    System.out.println(book.getTitle());
}
```

Esto funciona y no hay nada malo en el código, pero IntelliJ podría sugerirte utilizar un enfoque más optimizado:

```
library.getBooks().forEach(book -> System.out.println(book.getTitle()));
```

Estas sugerencias no solo te ayudan a escribir mejor código, sino que también te enseñan patrones idiomáticos en el lenguaje y los marcos que estás utilizando.

Visualización de la estructura del código

Otra parte importante para poder leer código de forma eficaz es comprender cómo está estructurado el código dentro de un proyecto o incluso de una sola clase. Los IDE suelen incluir funciones que visualizan la estructura de tu código. Si quieres intentar comprender cómo está arquitectado el código de la figura 2-11, por ejemplo, puedes mirar la estructura del proyecto y ver rápidamente que se trata de una aplicación monolítica y que el código está organizado en un arreglo de paquetes por características.

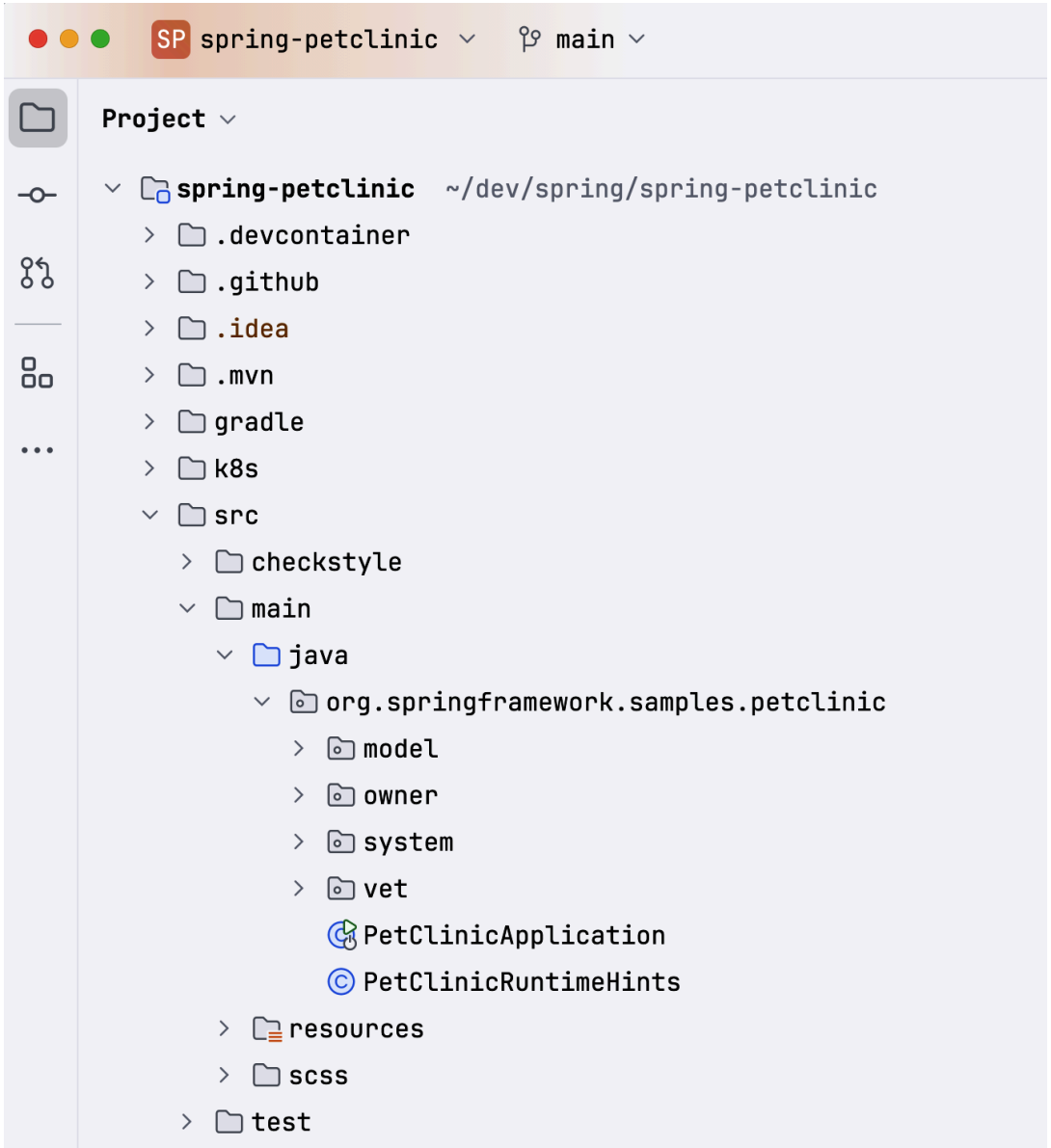


Figura 2-11. La estructura del proyecto puede ayudarte a ver el panorama general

Supongamos que intentas comprender qué métodos están disponibles en una clase determinada. Por supuesto, podrías consultar la documentación de la API, pero ¿sabes siquiera si se trata de la versión más actualizada de la documentación? Podrías abrir manualmente un archivo de clase y empezar a desplazarte por el código fuente, pero esto

podría llevarte bastante tiempo. La mayoría de los IDE tienen una forma de visualizar la estructura de una clase o componente. En el ejemplo que se muestra en la Figura 2-12, estás viendo la estructura de `OwnerController` en un proyecto llamado PetClinic. Puedes ver de un vistazo qué métodos están disponibles en la clase, lo que te da una idea rápida del propósito de esta clase y su funcionalidad.

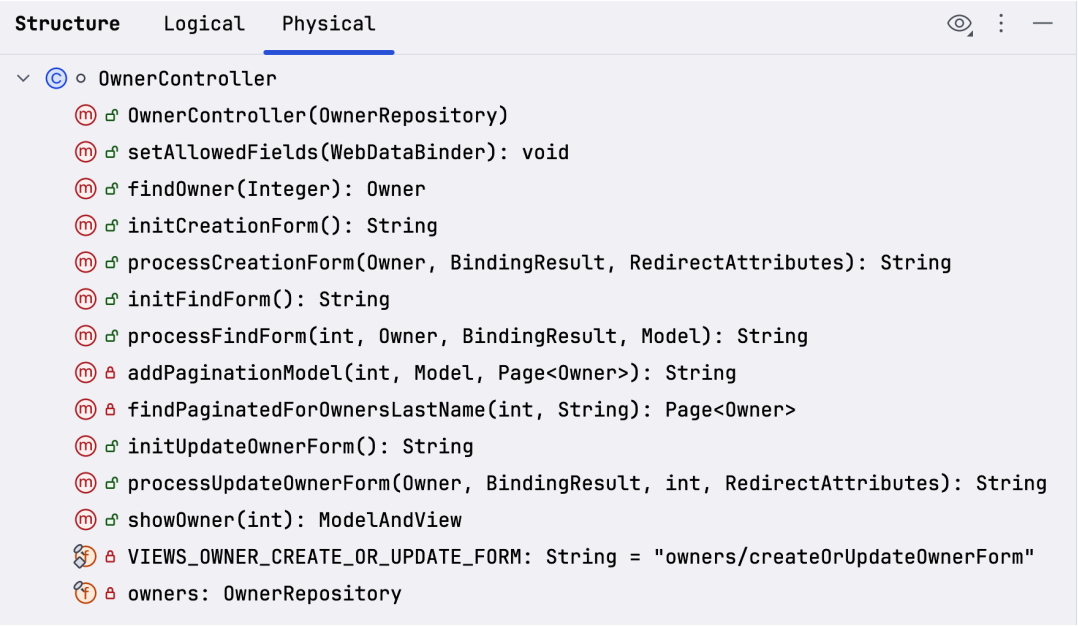


Figura 2-12. Comprensión de la funcionalidad de una clase de un vistazo con la vista de estructura

Leer pruebas para obtener información

Cuando se trata de explorar una nueva base de código, las pruebas sirven como otra forma de documentación para los desarrolladores. Las pruebas bien escritas revelan no solo lo que hace el código, sino también por qué lo hace de cierta manera. Mientras que el código te dirá

cómo funciona algo, las pruebas te dirán cómo se supone que debe funcionar.

Las pruebas como documentación viva

A diferencia de la documentación tradicional de , que a menudo queda obsoleta, las pruebas deben seguir siendo funcionales para superar las comprobaciones de integración continua. Esto las convierte en una fuente fiable y actualizada de información veraz sobre el comportamiento del sistema.

Las pruebas documentan las entradas y salidas esperadas de los métodos, las interacciones entre los componentes y el comportamiento general del sistema. Al leer primero las pruebas, puedes comprender lo que se supone que debe hacer un componente antes de profundizar en cómo lo hace realmente. Esto proporciona un contexto crucial que facilita la comprensión de la implementación.

La siguiente prueba verifica que los clientes habituales reciben un descuento por volumen del 10 % cuando realizan pedidos grandes por encima de un umbral determinado, lo que garantiza que el servicio de precios calcula correctamente los descuentos en función del tamaño del pedido y no del tipo de cliente:

```
@Test
public void shouldApplyBulkDiscountForLargeOrders() {
    // Given
    Customer regularCustomer = new Customer(CustomerType.REGULAR);
    Order bulkOrder = new Order(regularCustomer, 500.00);
```

```
        // When
        double finalPrice = pricingService.calculateFinalPrice(bulkOrder);

        // Assert
        assertEquals(450.00, finalPrice, 0.01);
    }
```

Con ese contexto, ahora puedes profundizar en la clase del servicio de precios y examinar el método « `calculateFinalPrice` » para comprender el código que produce ese resultado.

Comprender los flujos de trabajo y los procesos

Las pruebas suelen corresponderse con las historias de los usuarios, lo que las convierte en una forma excelente de comprender los procesos empresariales o los flujos de trabajo. Si se alejan y observan las pruebas de integración, a menudo pueden mostrarles una visión general de cómo funcionan los componentes juntos para cumplir los requisitos empresariales.

Busca pruebas que verifiquen procesos completos de principio a fin. Estas pruebas suelen contener código de configuración que representa escenarios del mundo real, lo que te proporciona contexto sobre cómo se comporta el sistema en producción.

La siguiente prueba verifica que, cuando un usuario envía una reseña de un producto, el servicio de reseñas actualiza correctamente la calificación media del producto y envía una notificación al administrador,

lo que garantiza que tanto la persistencia de los datos como los aspectos de notificación del proceso de reseñas funcionan según lo esperado:

```
@Test
public void submitProductReviewShouldUpdateRatingAndNotifyAdmin() {
    // Given
    Product product = productService.findProductById("XYZ789");
    User user = userService.findByUsername("janedoe");
    Review review = new Review(user, product, 4, "Great quality product!");

    // When
    ReviewResult result = reviewService.submitReview(review);

    // Then
    assertTrue(result.isSuccessful());
    assertEquals(4.2, productService.findProductById("XYZ789")
    .getAverageRating(), 0.01);
    verify(notificationService).sendReviewNotification(anyObject(), eq("XYZ789"));
}
```

Con este contexto, ahora comprendes todo el flujo de lo que sucede cuando un usuario envía una nueva reseña.

Descubrimiento de casos de perímetro y límites

En el desarrollo de software, un*caso límite e e* es una situación que se produce en el perímetro extremo de la entrada, las condiciones de funcionamiento o los patrones de uso esperados de un programa, y a menudo es el caso en el que se rompen las suposiciones. Estas entradas o

condiciones son válidas pero inusuales, y revelarán errores que no suelen producirse durante los escenarios «típicos» o «felices». En los códigos base existentes, es posible que encuentres más pruebas relacionadas con estos casos, ya que se han descubierto con el tiempo y se ha escrito una prueba para garantizar que no vuelvan a aparecer.

Presta especial atención a las pruebas con nombres como « `shouldHandleEmptyList` », « `shouldRejectInvalidInput` » o « `shouldTimeoutAfterTenSeconds` ». Estas pruebas te muestran los límites de las capacidades del sistema y resaltan los posibles puntos de fallo.

La siguiente prueba revela que el sistema gestionará las tarjetas de crédito caducadas y fallará correctamente. Esto es algo que podría no resultar tan obvio si solo leyeras la implementación:

```
@Test
public void shouldRejectPaymentWhenCreditCardIsExpired() {
    // Given
    CreditCard expiredCard = new CreditCard("4111111111111111", "05/20");
    Order order = new Order(new Customer(), 50.00);

    // When
    PaymentResult result = paymentService.processPayment(order, expiredCard);

    // Then
    assertFalse(result.isSuccessful());
    assertEquals(PaymentFailureReason.EXPIRED_CARD, result.getFailureReason());
}
```

Con este contexto, puedes revisar la implementación del método de pago del proceso y buscar la funcionalidad que detectará este caso extremo .

Una vez que se comprende un proyecto y se lee el código de forma eficaz, es el momento de modificar el código. Pero, ¿cómo se puede hacer esto de forma segura? Trataremos este tema en la siguiente sección.

La práctica hace al maestro

Al final del día, sé indulgente contigo mismo. Las bases de código modernas suelen ser muy extensas. Una sola persona no puede comprenderlas en su totalidad, y ese no es el objetivo. Tus conocimientos irán aumentando con el tiempo. Repite el proceso a medida que te encuentres con nuevas partes de tu proyecto. Puede resultar intimidante, pero todos los desarrolladores han pasado por ello. ¡Todo irá bien!

¿Cómo mejorar tus habilidades para leer código? Por mucho que te dé miedo, practica la lectura de código. Hay muchas opciones de código abierto, bien escritas y disponibles públicamente en una variedad de lenguajes entre las que puedes elegir. No hay atajos; no puedes mejorar sin practicar. Con el tiempo se vuelve más fácil y ganarás velocidad.

¿QUÉ HAY DE LA IA?

Como cualquier disciplina, las herramientas de software siguen evolucionando. Desde los editores de texto basados en shell hasta los IDE con herramientas de refactorización integradas y la finalización de código IntelliSense, escribir código se ha vuelto más fácil. Por supuesto, el tamaño y la complejidad de las aplicaciones en las que trabajas también han aumentado, ¡así que quizás sea un empate! Con la llegada de cosas como GitHub Copilot y la IA generativa como ChatGPT, algunos incluso han sugerido que los desarrolladores pronto serán reemplazados. Los rumores sobre el fin de los desarrolladores suelen ser muy exagerados, desde COBOL hasta los lenguajes de programación de cuarta generación, pasando por diversas soluciones de «programación» de arrastrar y soltar que no han logrado reducir a gran escala la demanda de ingeniería de software.

Estas herramientas tienen cabida en tu caja de herramientas; por ejemplo, puedes preguntarle a ChatGPT qué hace un fragmento de código,[9] pero sigues necesitando comprender el contexto. La IA no conoce el propósito de una clase determinada, por lo que sigues necesitando leer el código, revisar la documentación y hablar con tus compañeros desarrolladores. La IA (todavía) no comprende completamente toda la base de código de una empresa, y el riesgo e e de alucinaciones es real. Aunque la IA puede ayudarte, en última instancia, *debes* comprender los matices y las relaciones entre el código repartido en múltiples repositorios.

Como le gusta decir a Dan, cuando utilizas la IA, tú eres el piloto, no el pasajero: confía, pero verifica. La IA puede comprender el código en el

vacío, pero es posible que no comprenda una base de código de nivel empresarial sin posibles alucinaciones. Las aplicaciones modernas suelen estar repartidas en muchos repositorios y aprovechan múltiples bibliotecas. Tú, como desarrollador, debes comprender las relaciones y las complejidades para que las herramientas de IA puedan acelerar con éxito el desarrollo.

Conclusión

Podría decirse que la programación se enseña al revés: aprendes a escribir antes de aprender a leer y, sin embargo, pasarás una parte importante de tu carrera leyendo código escrito por otra persona. Aunque puede que no disfrutes tanto del código existente como del desarrollo desde cero, te da de comer. En lugar de huir de la situación, aprende a aceptarla, ya que hay mucho que ganar profesionalmente. Sé consciente de los sesgos cognitivos. No tengas miedo de arremangarte y rebuscar en un código desconocido; aprenderás algo. A medida que tu comprensión crezca, deja el código mejor de lo que lo encontraste, facilitando el camino al próximo desarrollador e , ¡que podría ser tú mismo!

Ponerlo en práctica

Si quieres mejorar en la lectura de código, no hay atajos; necesitas leer más código. Por suerte, tienes a tu disposición una gran cantidad de

proyectos de código abierto. Reserva un par de horas para leer parte del código del marco que utilizas (o el que te gustaría utilizar) en el trabajo o en otro proyecto que te interese. Si no estás seguro de por dónde empezar, echa un vistazo a los repositorios más populares en GitHub. Aplica las técnicas que has aprendido en este capítulo. En un par de meses, elige otra parte del proyecto que hayas explorado o prueba uno completamente diferente; ¿te ha resultado más fácil que la primera vez? Sigue así; con el tiempo, tus habilidades para leer código mejorarán.

Trabajar con código existente es también una habilidad que hay que desarrollar y, una vez más, el software de código abierto te ofrece un enorme campo de juego para explorar. Contribuir al código abierto es un excelente laboratorio de aprendizaje, y empezar no es tan difícil como podrías pensar.[10] Elige un proyecto y dedica unas horas a trabajar en él.

Recursos adicionales

- «El código como diseño: tres ensayos», de Jack W. Reeves
- «Leer código es más difícil que escribirlo», de Trisha Gee
- «Leer el código de otras personas», de Patricia Aas
- «Cómo leer de forma rápida y eficaz el código de otras personas», de Alex Coleman
- «Cómo leer código sin tirarte de los pelos», de Sunny Beatteay

[1] A veces conocido como *«vibe coding»*.

[2] O, como dijo una vez Neal Ford: «Las buenas prácticas de hoy son los antipatrones de mañana»

[3] Michael I. Norton, Daniel Mochon y Dan Ariely, «The 'IKEA Effect': When Labor Leads to Love» (El «efecto IKEA»: cuando el trabajo conduce al amor), documento de trabajo de HBS, n.º 11-091 (2011), *https://oreil.ly/zuETL*.

[4] *Es posible que* dicha documentación esté desactualizada, confía pero verifica.

[5] Para obtener más información sobre los ADR, consulta el capítulo 3 de *Head First Software Architecture*, de Raju Gandhi et al. (O'Reilly, 2024).

[6] Por supuesto, no todas las aplicaciones tienen archivos con nombres tan claros; es posible que tengas que hacer un uso amplio de tus herramientas de búsqueda favoritas.

[7] Por ejemplo, ⌥⇧⌘U (macOS) / Ctrl-Alt-Shift-U (Windows/Linux) en IntelliJ IDEA generará un diagrama UML.

[8] Una interfaz Java es un tipo de referencia que define un contrato de métodos que deben proporcionar las clases de implementación, lo que permite la abstracción y el polimorfismo en los programas Java.

[9] Es probable que los abogados de tu organización tengan opiniones firmes sobre el uso de este tipo de herramientas, así que comprueba bien las políticas de tu empresa antes de pegar tu algoritmo de precios patentado en un modelo que podría estar utilizando tu código como datos de entrenamiento.

[10] Muchos proyectos tienen listas de errores marcados como «para colaboradores noveles», pero no tengas miedo de ponerte en contacto con los colaboradores

actuales del proyecto; la mayoría estará encantada de ayudarte a ponerte en marcha.